Date

Expt. No. 4 Page No. 13.

Aim:- To develop a Java Application in which a client communicates with a server using the java.net package & TCP sockets.

Theory:-

- Client :- A program that requests a service from the server.
- Server :- A program that waits for client requests & provide responses.
- Soket :- An endpoint used for communication between two programs over a network.
- TCP (Transmission Control Protocol) :- A connection-oriented & reliable protocol used to transfer data between client & server.
- Java provides ServerSocket to Accepts client connection & socket to establish communication

Code

File 1. server.java

```
import java.io.*;
import java.net.*;
public class Server {
    public static void main(String[] args) {
        try {
            ServerSocket server = new ServerSocket(5000);
            System.out.println("Server started...");
            System.out.println("Waiting for Client...");
            Socket socket = server.accept();
            System.out.println("Client Connected");
```

Teacher's Signature

Output (Server Terminal)

Server started...
Waiting for client...
Client connected.
Client Say: Hello Server

(Client terminal)
Message sent to server.
Server says: Hello Client

Date

Expt. No. _____ Page No. 14

```
BufferedReader input = new BufferedReader(
    new InputStreamReader(Socket.getInputStream
PrintWriter output = new PrintWriter(
    Socket.getOutputStream(), true);
String message = input.readLine();
System.out.println("Client says:" + message);
output.println("Hello Client");
socket.close();
server.close();
} catch (IOException e){
    System.out.println(e.getMessage());
}
}
```

File 2 Client.java

```
import java.io.*;
import java.net.*;
public class Client {
    public static void main(String[] args){
try {
    Socket socket = new Socket("Localhost", 5000);
BufferedReader input = new BufferedReader(
    new InputStreamReader(socket.getInputStream
PrintWriter output = new PrintWriter(
    socket.getOutputStream(), true);
output.println("Hello Server");
System.out.println("Message sent to server");
String response = input.readLine();
System.out.println("Server says:" + response);
socket.close();
}
```

Teacher's Signature _____

```
        catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Viva Question

Q1 What is client & server architecture?
A1 It is a model in which a client requests services & a server processes the request & returns a response.

Q2 What is a socket in Java?
A2 A socket is an endpoint that allows two programs to communicate over a network.

Q3 What is the difference between Socket & ServerSocket?
A3 Socket is used by the client to connect to a server, while ServerSocket waits for & accepts client connections.

Q4 What is the use of port number 5000?
A4 Port 5000 identifies the server application that listeners for incoming client connections in this program.

Q5 What is the difference between TCP & UDP?
A5 TCP is connection-oriented & provides reliable, ordered data delivery. UDP is connectionless & does not gaurantee delivery or order.

Date

Expt. No. 5 Page No. 16

Aim:- To develop a java RMI application in which a client invokes a method of a remote object to perform addition of two numbers.

Theory

- RMI (Remote method invocation) allows a Java program to call method of an object running in another Java virtual method (JVM)
- Remote Interface :- Defines the method that can be called remotely.
- Remote Object : Implements the remote interface & performs the actual operation.
- RMI Registery :- Stores the name & reference of remote objects so clients can locate them.
- The client looks up the remote object, calls its method & receives the result from the server.

Code :- File 1 MyInterface.java.

```
import java.rmi.*;
public interface MyInterface extends Remote{
    int add(int a, int b)
    throws RemoteException ;
}
```

File 2 MyRemote.java

```
import java.rmi.*;
import java.rmi.server.*;
public class MyRemote extends UnicastRemoteObject
    implements MyInterface{
```

Teacher's Signature

Output

Run commands:

javac *.java

java Server

RMI Server is ready

java Client

Sum = 30

Date

Expt. No. _____ Page No. 17

```
public MyRemote() throws RemoteException {
    super();
}
public int add(int a, int b) throws RemoteException {
    return a+b;
}
}
```

File 3 Server.java

```
import java.rmi.*;
import java.rmi.registry.*;
public class Server {
public static void main(String[] args){
try {
    LocateRegistry.createRegistry(1099);
MyInterface obj = new MyRemote();
Naming.rebind("rmi://localhost/Add", obj);
System.out.println("RMI Server is ready");
} catch (Exception e) {
    System.out.println(e);
}
}
}
```

File 4. Client.java

```
import java.rmi.*;
public class Client {
public static void main(String[] args){
try{ MyInterface obj =
    (MyInterface) Naming.lookup("rmi://localhost/A
int result = obj.add(10, 20);
System.out.println("Sum = " + result); }
catch (Exception e) {
System.out.println(e); }
}
```

Teacher's Signature

Date

Expt. No. Page No. 18.

Viva Question

Q1 What is RMI?

Ans RMI stands for Remote Method Invocation. It allows a java program to call a method of an object running in another JVM.

Q2 What is a Remote Interface?

Ans A Remote Interface defines the methods that can be called remotely. It must extends java.rmi.Remote.

Q3 What is a ~~Remote Interface~~? RMI Registery?

Ans The RMI Registry is a naming service that stores references to remote objects so clients can find them.

Q4 What is the use of Naming.lookup()?

Ans Naming.lookup() searches the RMI Registry for a remote object and returns its references to the clients.

Q5 What is RemoteException?

Ans RemoteException handles communication-related errors that may occurs during remote method calls.

Teacher's Signature

Date

Expt. No. 6. Page No. 19

Aim:- To install & Configure Apache Tomcat & create & run a simple web application using an HTML page.

Theory:-

- Apache Tomcat is a web server & servlet container used to run java web Applications.
- A web application is accessed through a web browser using a URL.
- The webapps directory is the default location for web applications in Tomcat. We create a folder named MyWebApp & place an index.html file inside it.
- Tomcat processes the browser's HTTP request & serves the HTML page as an HTTP response.

Code:- Create following folder structure -

```
apache-tomcat/
  ↳ webapps/
      ↳ MyWebApp/
          ↳ index.html
```

File Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title> My Web Application </title>
</head>
<body> <h1> Welcome to My web Application </h1>
    <p> This Application is Running on Apache Tomcat </p>
</body>
</html>
```

Teacher's Signature

Output

Welcome to My Web Application

This application is running on Apache Tomcat

Expected output displayed in Browser

Date

Expt. No.

Page No. 23.

Procedure

1. Download & Extract Apache Tomcat.
2. Open the Tomcat bin directory.
3. Start server using startup.bat on windows
4. Create a folder named MyWebApp inside the webapps directory.
5. Create the index.html file inside MyWebApp & write the above HTML code.
6. Open a browser & enter the following URL: http://localhost:8080/mywebApp/

Viva Question

Q1 What is Apache Tomcat?

Ans Apache Tomcat is a web server & servlet container used to run Java Lab web applications.

Q2 What is the webapps folder?

Ans It is the default Tomcat directory where web applications are stored & deployed.

Q3 What is localhost?

Ans localhost refers to the computer on which the server is running.

Q4 What is Port 8080?

Ans Port 8080 is the port used by Tomcat in this experiment to receive HTTP request.

Q5 What is the Purpose of index.html?

Ans index.html is the HTML page displayed when the web application is opened through its URL.